

8 - Analisi dei requisiti

Modello di dominio

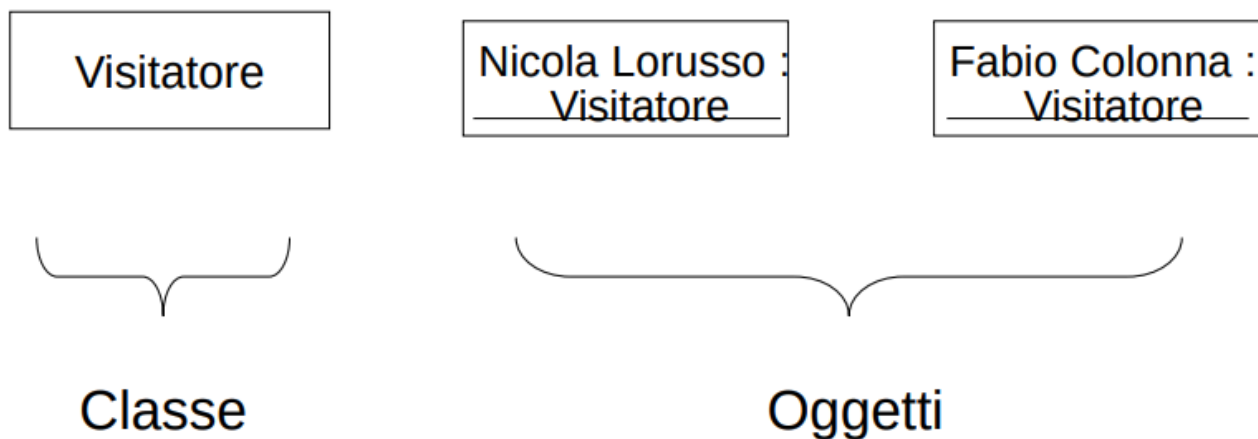
Il modello di dominio è utile per comprendere e comunicare i concetti fondamentali di un dominio (quindi di un ambito).

Per rappresentarlo si utilizza un **diagramma delle classi con prospettiva concettuale**, il quale rappresenta in modo visivo i concetti caratteristici del dominio che stiamo studiando. Inoltre questo modello è **indipendente dal software** ed è chiamato anche **modello concettuale**.

Classe come concetto

Una classe rappresenta un concetto ben definito che permette l'astrazione di istanze simili.

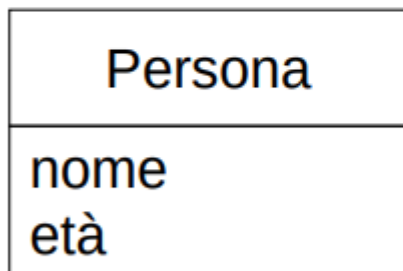
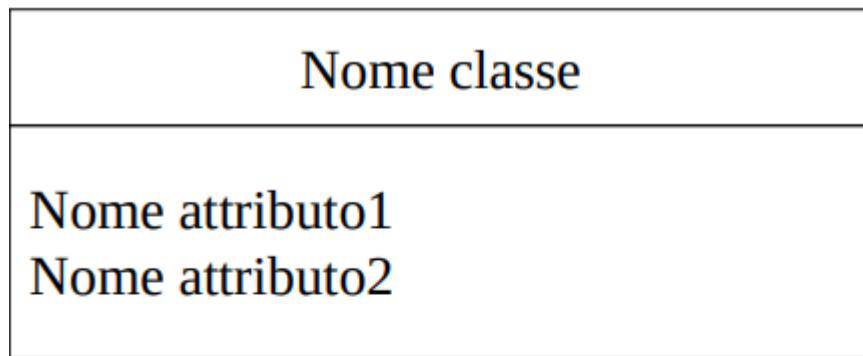
— Es. Visitatore, Abbonato, Prodotto, Vendita



Le proprietà della classe sono gli **attributi** e le **associazioni**.

Attributi e Associazioni

Un **attributo** è una proprietà rilevante di un concetto (o classe) che contiene un valore per ogni oggetto.



Classe con attributi

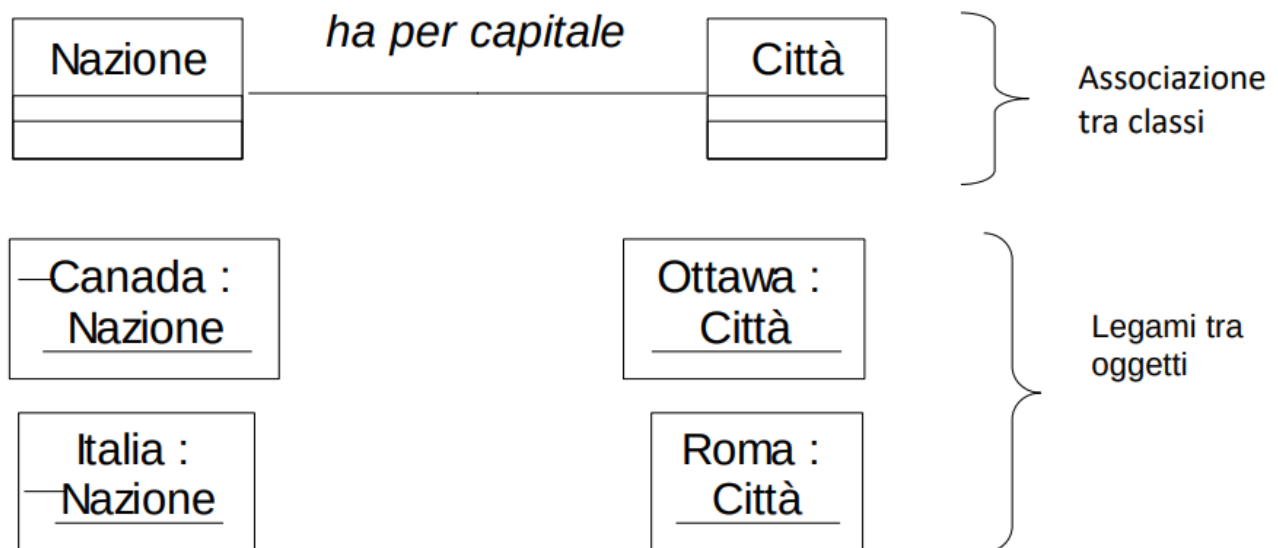
Un'**associazione** è una relazione rivelante tra due concetti (o classi) ed è etichettata.

Le associazioni sono generalmente bidirezionali, anche se generalmente vengono lette da sinistra a destra o dall'altro al basso.

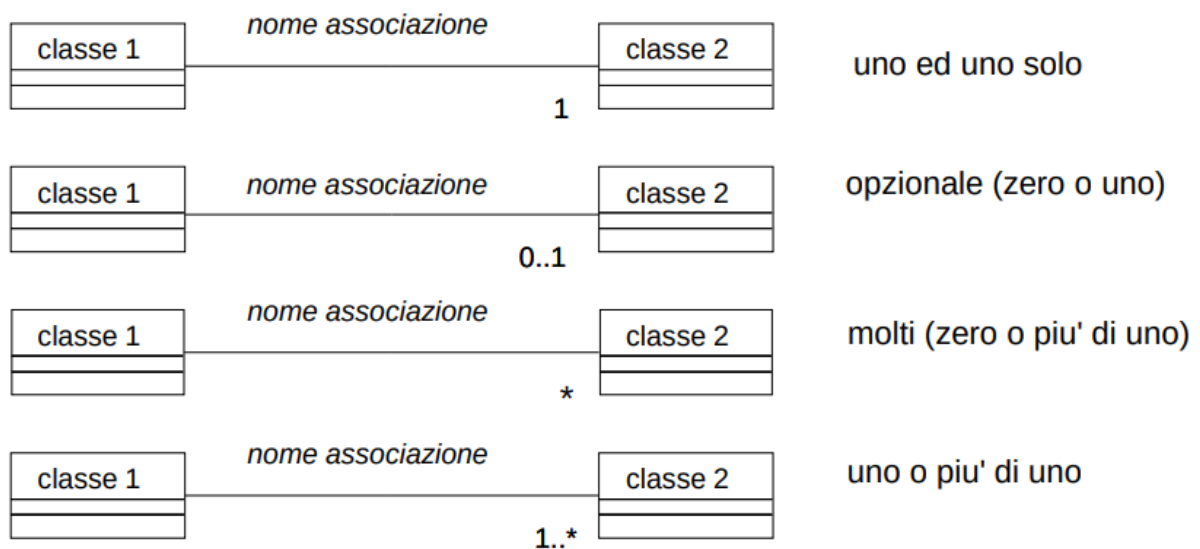
Le associazioni possono essere anche **ricorsive**.

Legami delle associazioni

Un'associazione può rappresentare in modo più preciso anche un **gruppo di legami** tra gli oggetti delle classi associate:



Prof. Filippo Lanubile



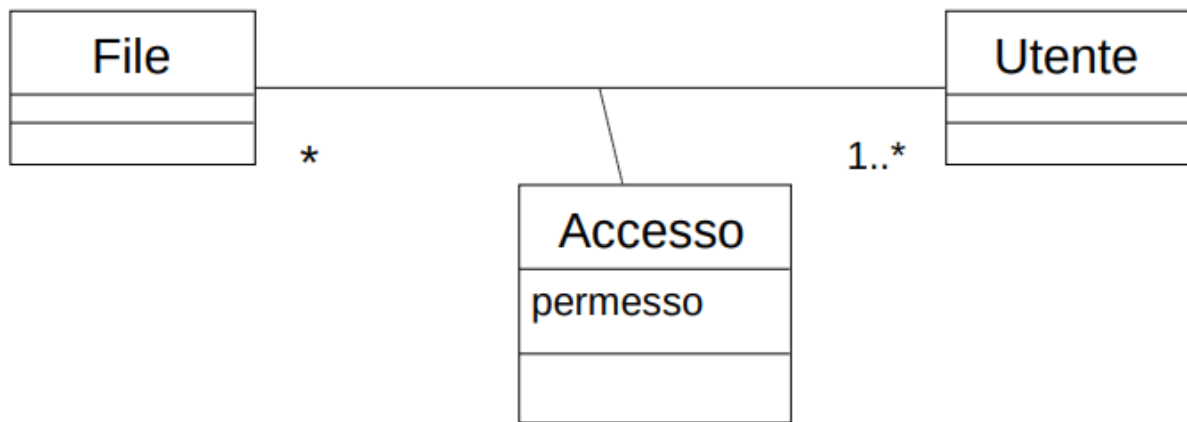
Prof. Filippo Lanubile

La molteplicità specifica quanti oggetti di una classe possono essere associati con un singolo oggetto di un'altra classe

Classi associative e attributi di legame

Le classi nate dalle associazioni sono utilizzate per modellare proprietà intrinseche nella relazione delle due classi coinvolte. Un attributo di una classe associativa contiene un

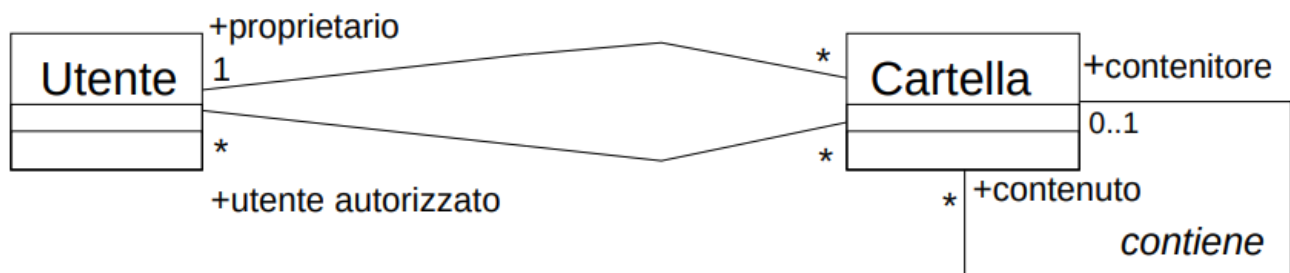
valore per ogni legame. Molto spesso nascono relazioni N-N (molti a molti)



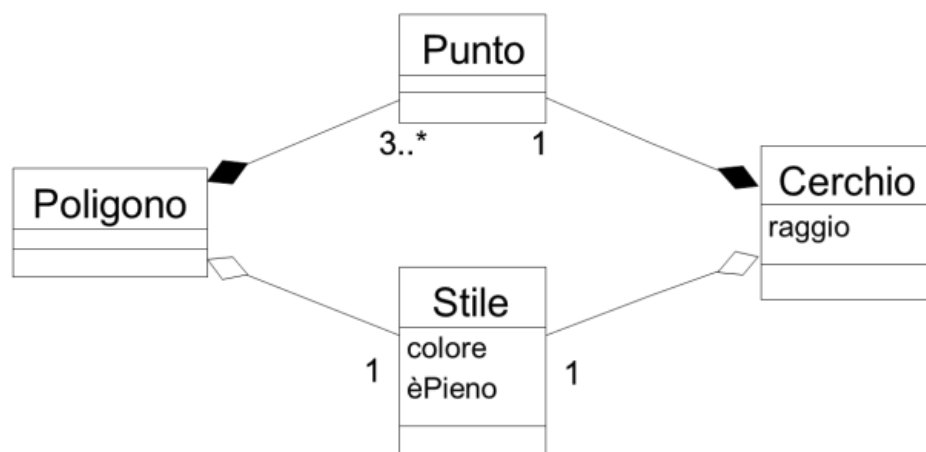
Ruoli

Un'alternativa grafica per non inserire l'etichetta nell'associazione è quella di definire dei **ruoli**. I ruoli rappresentano il nome della proprietà e sono posti alla fine della linea di congiunzione.

I ruoli hanno gran efficacia nelle associazioni tra oggetti della stessa classe, come l'esempio precedente:



L'aggregazione del contenuto e del contenitore viene rappresentato anch'esso graficamente tramite il **daimo**, il quale può essere **vuoto** o **pieno**:



La **composizione** è una forma definita **forte** dell'aggregazione, poiché stiamo definendo che:

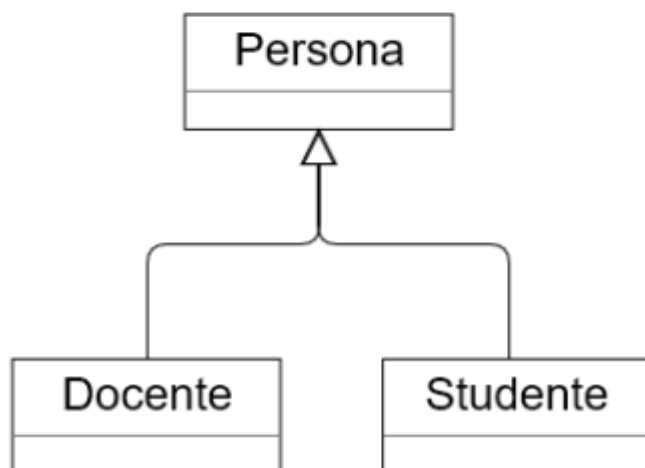
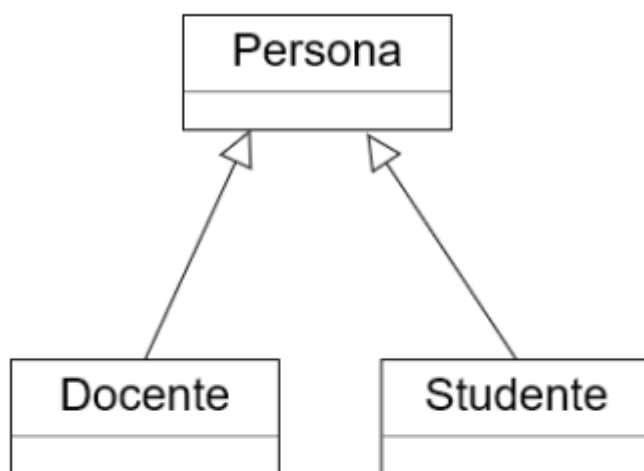
- La parte componente non può esistere senza il contenitore

- La parte componente può appartenere solo ad un unico oggetto contenitore
La composizione è una forma definita **debole** invece quando l'oggetto può esistere anche senza un'altra classe.
In lettura queste due parti si leggono con **è parte di o al contrario è composto da**.

Generalizzazione

L'associazione tra classi viene definita sotto il concetto di **è un tipo di** o al contrario come **può essere un**.

Questo concetto è utilizzato per definire le **tassonomie** di classi, così possiamo dividere le classi in **superclasse** e le loro rispettive **sottoclassi**.



Per definizione, ogni oggetto di una sottoclasse è anche un oggetto della sua superclasse.

⚠ Attenzione

Non tutti gli attributi sono generali per tutte le sottoclassi di una superclasse.

I requisiti

Un requisito è una caratteristica o condizione che un sistema è tenuto a rispettare. La formulazione dei requisiti chiarisce il modo in cui i bisogni dei clienti o degli utenti dovranno essere soddisfatti dai progettisti. L'insieme dei requisiti è noto come **spazio delle soluzioni ammissibili**.

I requisiti si dividono in due tipi:

- **Requisiti funzionali o features**
- **Requisiti non funzionali:**
 - affidabilità
 - efficienza
 - usabilità
 - manutenibilità
 - portabilità

I requisiti devono essere espressi in maniera corretta, dettagliata e chiara nel modo più possibile ed in più ogni metodo implementato per la soddisfazione di questi requisiti devono essere testabili.

Lo scambio dei requisiti fino al prodotto finito avviene nel seguente schema:



Sebbene lo schema sia tipico di altri contesti ingegneristici, l'applicazione nel campo del software è spesso problematica.

Problemi nella raccolta dei requisiti

Idealmente si dovrebbero seguire questi passaggi semplici:

- Il committente comunica i requisiti alla partenza del progetto

- Il compito degli analisti è soltanto quello di acquisirli e di comprenderli

Concettualmente e realmente non avviene mai questo, poiché il cliente non conosce bene i termini da utilizzare per far comprendere appieno all'analista cosa raccogliere come dati, ma può essere astratto, confuso e non preciso o diverso da come se lo aspettava il cliente a prodotto aggiornato nello sviluppo.

Si eseguono più step nella scoperta dei dati e specialmente durante l'evoluzione del progetto, poiché nasceranno più requisiti:

- I requisiti possono nascere, o cambiare, anche dopo il raggiungimento dell'accordo (o la stipula del contratto), mentre sono in corso le attività di realizzazione
- Fattori che possono determinare un cambiamento di requisiti:
 - Innovazioni legislative
 - Cambiamenti negli scenari di mercato o nelle strategie aziendali,
 - Nuove opportunità e bisogni
 - Scoperta di errori

Ingegneria dei requisiti

Esistono varie tecniche per:

- **Scoprire** i requisiti
- **Specificare** i requisiti
- **Validare** i requisiti
- **Gestire** dei cambiamenti

Queste tecniche sono notate come l'**ingegneria dei requisiti**.

Scoprire i requisiti

Si creano delle **interviste** fisicamente tra analista e cliente o tramite **workshop** dove molti membri del team parlano con molti clienti coinvolti.

La figura che identifica gli interlocutori è chiamato **stakeholder**.

Si generano **prototipi UI** da far vedere al cliente per poter continuare a procedere lo sviluppo software e avviene anche il recupero e l'analisi della documentazione da sistemi esterni.

SRS

La specifica dei requisiti o **SRS** avviene tramite la sottoscrizione di **documenti**, la cui struttura può dettata dagli standard come IEEE Std 830-1998...

Il documento è suddiviso in varie parti:

1. **Introduzione**
2. **Descrizione**
3. **Requisiti specifici**

- Requisiti funzionali
- Requisiti non funzionali

4. Appendici

5. Indice

Stile di descrizione dei requisiti

I requisiti si descrivono tramite le:

- **Proposizioni**, sia funzionali che non
- **Casi d'uso**, usati solo per quelli funzionali
- **User stories**, usati solo per requisiti funzionali nello sviluppo agile

Casi d'uso e user stories

Essi rappresentano una funzionalità che il sistema deve offrire ai suoi **attori**.

I casi d'uso differiscono dalle user stories dal **metodo in cui sono descritti**:

- **User story**: descritte con brevi annotazioni da post-it
- **Casi d'uso**: descritti tramite la narrazione delle interazioni tipiche tra attori e sistema

L'attore è il ruolo dell' utilizzatore del prodotto, che può essere un utente del prodotto finale o utilizzatore di mezzo, o anche qualcuno che svolge interazioni con il sistema.

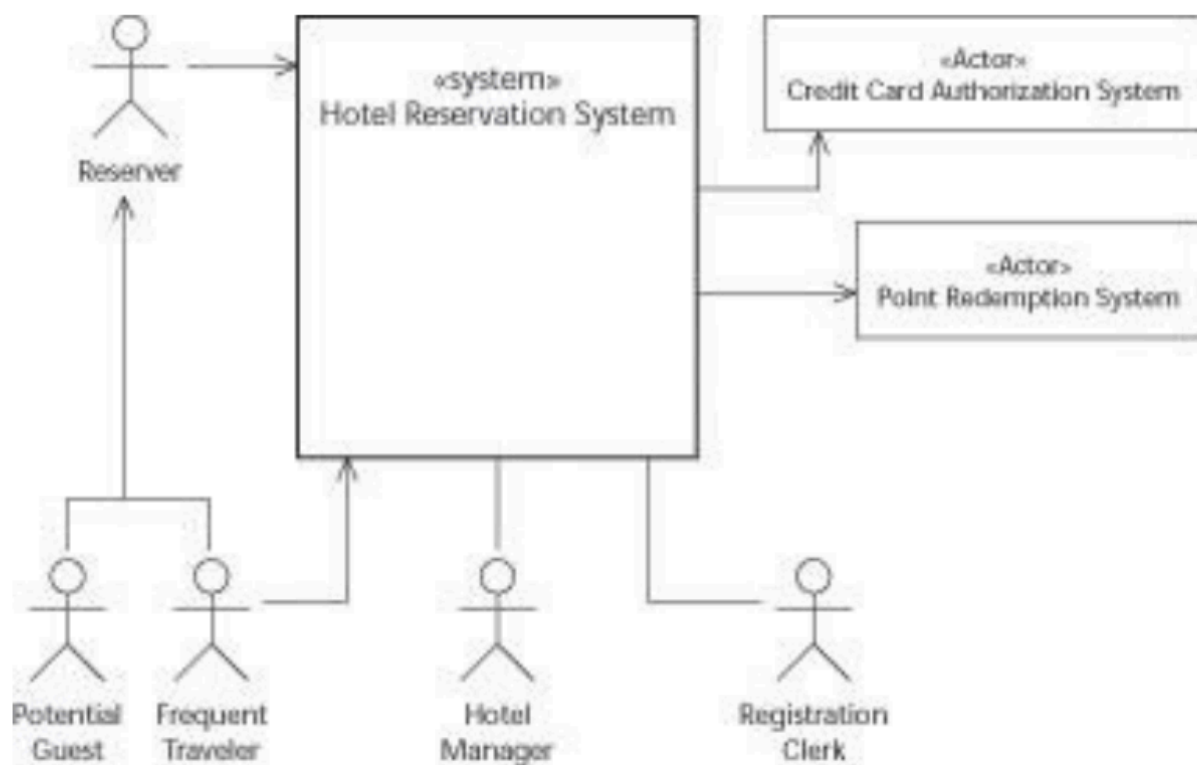
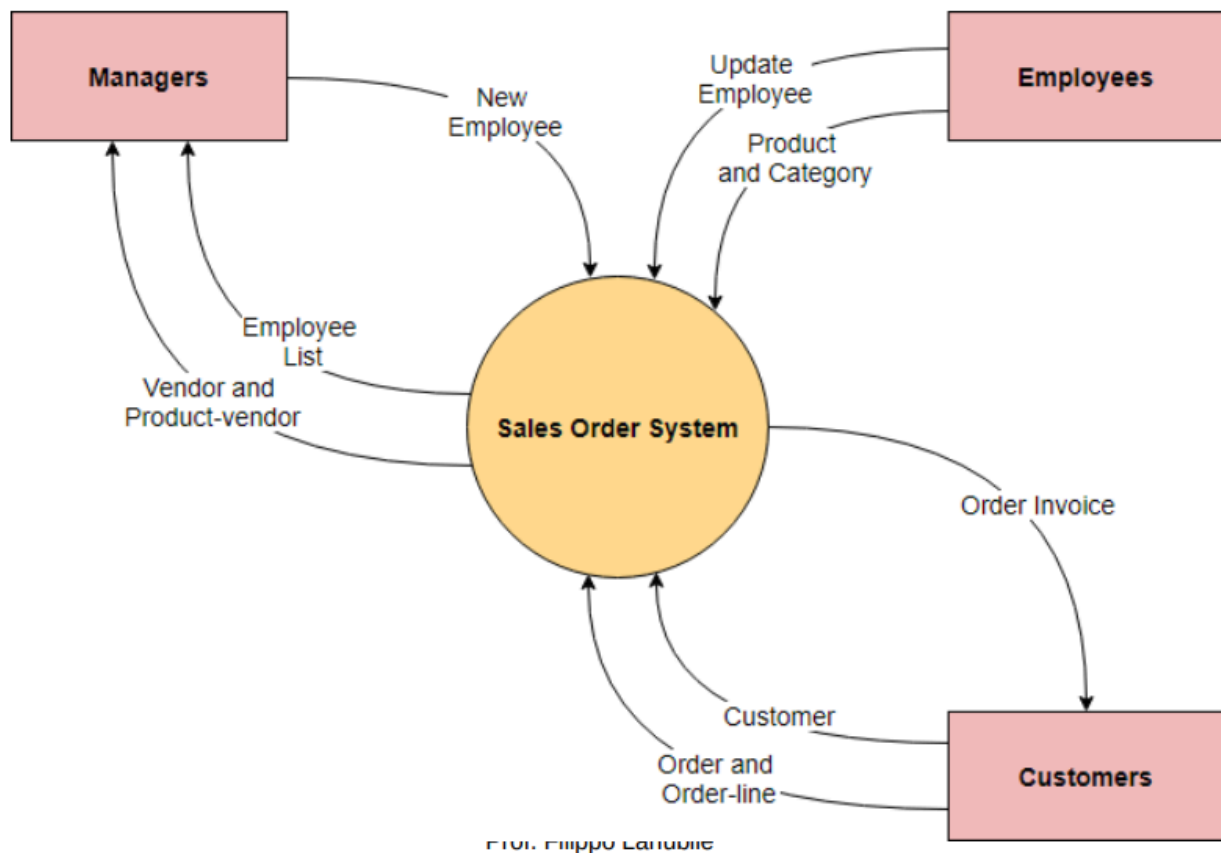
L'attore è definito **esterno al sistema** e può essere:

- Una persona fisica
- Sistema che lavora esternamente al progetto (come il sistema di contabilità)
- Dispositivo hardware esterno

L'attore può essere anche riferito al plurale

Diagramma di contesto

Il diagramma di contesto ci permette di definire in modo grafico i confini del sistema e chiarisce anche le entità che devono interagire con quest'ultimo.



Descrizione caso d'uso

La descrizione del caso d'uso si divide in

- **Formato breve:** composto da un singolo paragrafo di poche righe
- **Formato strutturato:** composto da scenari di interazione

Scenari di interazione per casi d'uso

I casi d'uso possono essere descritti sotto forma di scenari di interazione (dialogo) tra gli attori e il sistema

In generale si dividono in:

- Scenario principale di successo
- Scenari alternativi ed eccezionali espressi come estensioni

Uno scenario definisce cosa accade nel sistema in seguito all'evento di innesco, ossia:

- Come e quando il caso d'uso inizia
- Chi inizia il caso d'uso
- Interazione tra attore/i e caso d'uso, insieme a cosa viene scambiato
- Come e quando il caso d'uso termina

Esempio di descrizione di caso d'uso mediante scenari

Codice Caso d'uso: UC05

Nome: Accendere un conto corrente

Attori: sportellista bancario

Obiettivo: Permette a uno sportellista bancario di accendere un conto corrente presso la propria filiale, per conto di un cliente richiedente

Pre-condizioni: Il conto corrente non deve ancora esistere; gli intestatari del conto devono essere già censiti

Post-condizioni: Il conto corrente deve essere aperto, con riferimento a intestatari censiti in anagrafica

Scenario Base:

1. Lo sportellista richiama la transazione di inserimento nuovo conto
2. Il sistema propone una videata che richiede i codici cliente degli intestatari del conto
3. Lo sportellista fornisce i codici degli intestatari
4. Il sistema visualizza le anagrafiche corrispondenti e richiede le condizioni da applicare al conto
5. Lo sportellista specifica le condizioni del conto (standard, legate a convenzione, derogate) e chiede l'inserimento
6. Il sistema stampa il contratto con il numero assegnato al conto

Estensioni

3a Se lo sportellista non conosce i codici:

3a.1 Lo sportellista richiama la transazione di ricerca anagrafica (<<include>> UC14 Ricercare anagrafica)

4a Se il sistema non riconosce un codice cliente, o se fornisce in risposta una anagrafica imprevista

4a.1 Lo sportellista può correggere il codice cliente, o terminare il caso d'uso annullando l'inserimento

5a Se lo sportellista non conosce la convenzione da associare a conto per definire le condizioni

5a.1 Lo sportellista richiama la transazione di ricerca convenzione (<<include>> UC21 Ricercare convenzione)

Altri requisiti:

L'accensione di un conto corrente può avvenire nell'ambito temporale di apertura della specifica filiale (REQ16)

Validazione dei requisiti

Nella validazione dei requisiti l'obiettivo è assicurarsi che la specifica dei requisiti del software rappresenti in modo accurato e chiaro i bisogni del committente.

La revisione dei requisiti avviene mediante ispezioni ottenute in diversi modi:

- Lettura individuale e discussione in gruppo
- Utilizzo di checklist
- Difetti classificati come:
 - Omissione
 - Informazione ambigua
 - Inconsistenza
 - Fatto non corretto
 - Informazione estranea

Gestione dei cambiamenti dei requisiti

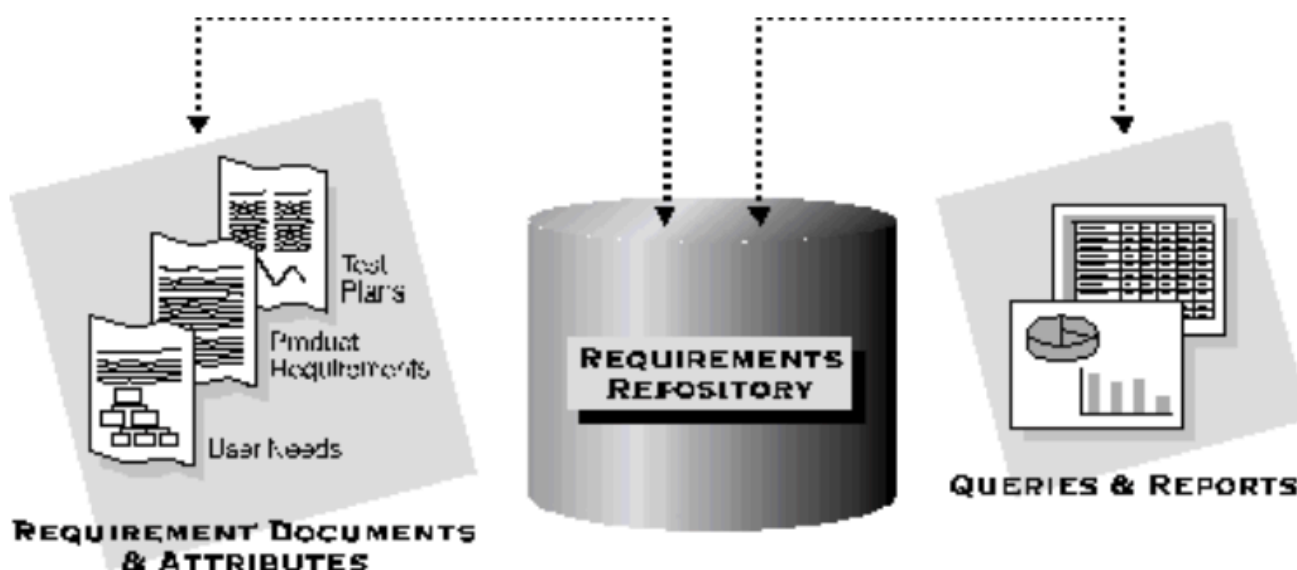
Per gestire i cambiamenti dei requisiti sono fondamentali 4 punti:

- Minimizzazione delle ridondanze
- Tracciabilità
- Annotazione degli attributi dei requisiti
- Procedure di gestione della configurazione

Minimizzazione delle ridondanze

aspetto fondamentale dell'analisi e della definizione dei requisiti software consiste nella minimizzazione delle ridondanze, possibile tramite degli strumenti specializzati.

Questi strumenti sono fondamentali per l'intera architettura dell'informazione di progetto, mettendo in comunicazione i bisogni primari degli utenti (User needs) con i documenti dei requisiti e i relativi attributi. Inoltre, l'utilizzo di un repository centralizzato permette di estrarre dati in modo strutturato attraverso la generazione di interrogazioni (queries) e reportistica dedicata



Tracciabilità

La qualità di un documento di Specifica dei Requisiti Software (SRS) dipende fortemente dalla sua **tracciabilità**.

Un SRS è considerato tracciabile qualora l'origine di ciascun requisito in esso contenuto risulti chiara ed inequivocabile, e qualora il documento stesso faciliti il costante riferimento a ogni singolo requisito durante tutte le successive fasi di sviluppo e di manutenzione del prodotto

Annotazione degli attributi dei requisiti

Per una corretta sistematizzazione e prioritizzazione, è indispensabile procedere con l'annotazione puntuale degli attributi associati a ciascun requisito. Tali attributi permettono di qualificare il requisito secondo diverse dimensioni analitiche.

Si dividono in diverse annotazioni:

- Annotazione della criticità dei requisiti:
 - Se malfunzionamenti possono causare problemi gravi (sicurezza persone, perdite finanziarie, ...)
- Annotazione dell'importanza dei requisiti:

- obbligatori (must)
- desiderabili (should)
- opzionali (may)
- Annotazione della stabilità dei requisiti:
 - Stabili
 - Volatili
- Annotazione dello stato dei requisiti:
 - richiesti (da chi?)
 - validati
 - realizzati (in quale release?)
 - rimossi (quando?)

Gestione della configurazione

La gestione della configurazione comprende un insieme di procedure volte a garantire il controllo assoluto sulle modifiche apportate ai requisiti e agli artefatti di progetto.

Queste procedure sono volte al:

- Permettere solo che le persone autorizzate possano effettuare cambiamenti
- Non essere possibile effettuare cambiamenti in modo concorrente
- Tenere traccia dei cambiamenti in modo tale poter tornare indietro
- Tenere traccia di chi ha effettuato il cambiamento e delle motivazioni

User story

Nell'ambito dell'ingegneria del software moderna, la descrizione funzionale avviene frequentemente tramite l'utilizzo delle **user story**

Template delle user story

Template

In qualità di <ruolo utente>,

voglio <obiettivo>

[in modo tale da <motivo>]

Esempio

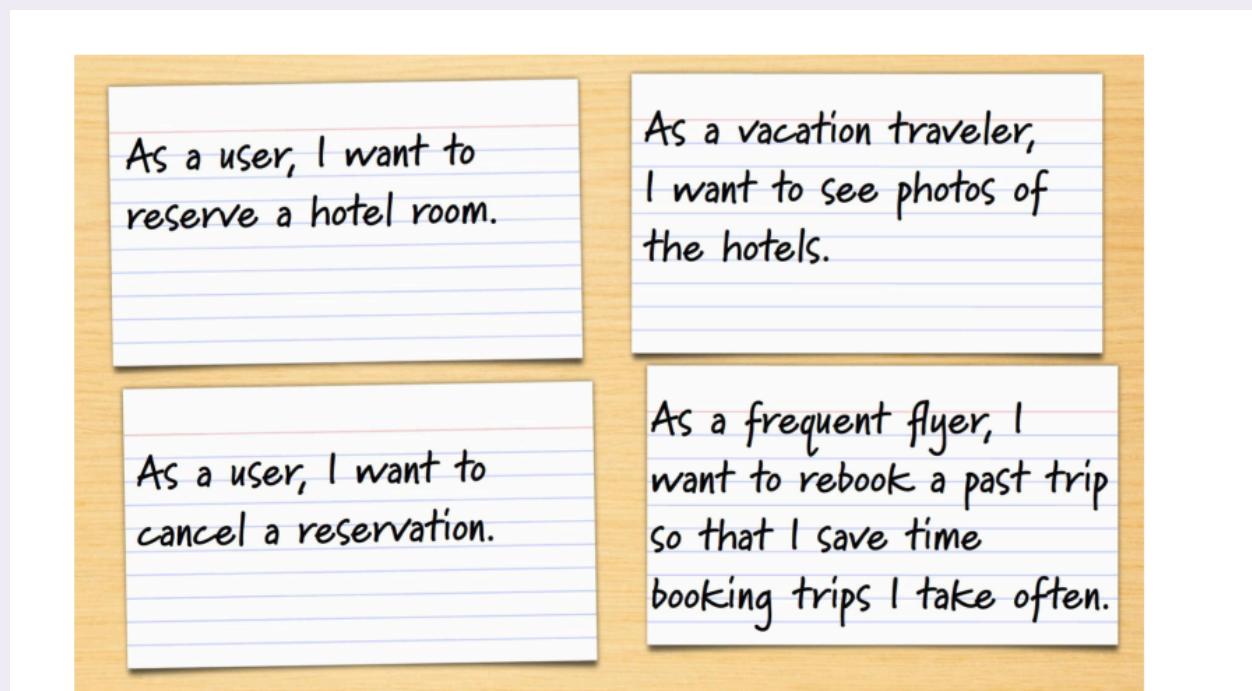
In qualità di *cliente occasionale*,

voglio *visualizzare le foto dell'albergo selezionato*

in modo tale da *decidere se effettuare la prenotazione*

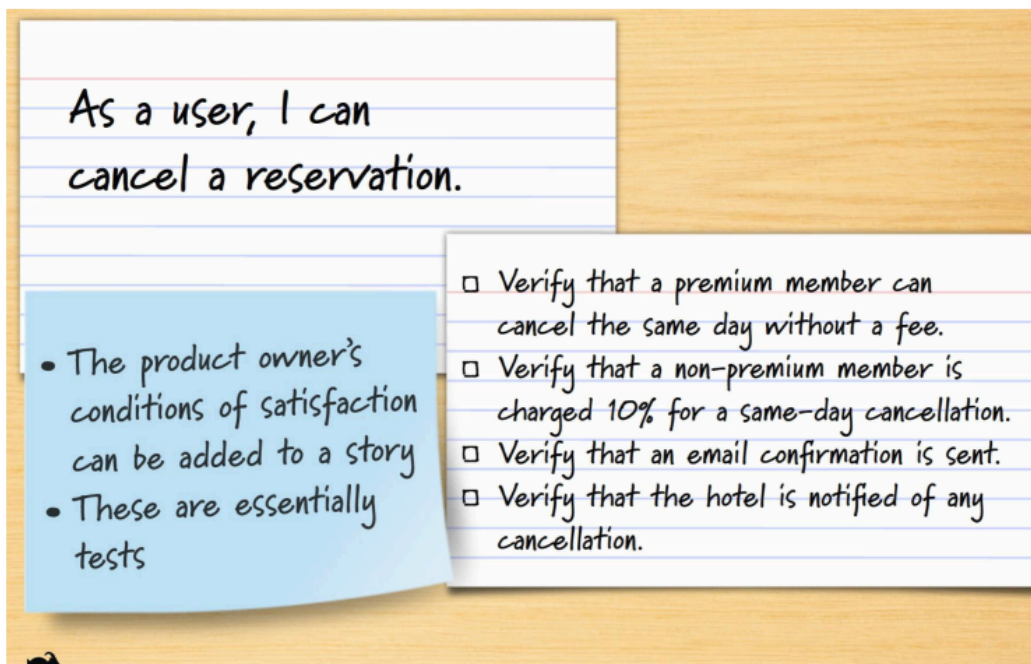
- **Card:** Abbastanza breve da poter essere scritta su un post-it
- **Conversation:** Abbastanza per avviare la discussione per chiarire i dettagli
- **Confirmation:** A fare da specifica sono i criteri (test) di accettazione

☰ Esempio di user-story

**Test di accettazione e specifiche**

I test di accettazione costituiscono la vera e propria specifica tecnica e funzionale di una user story. Le condizioni di soddisfazione, generalmente dettate dal Product Owner, vengono aggiunte direttamente alla storia e si traducono essenzialmente in test eseguibili

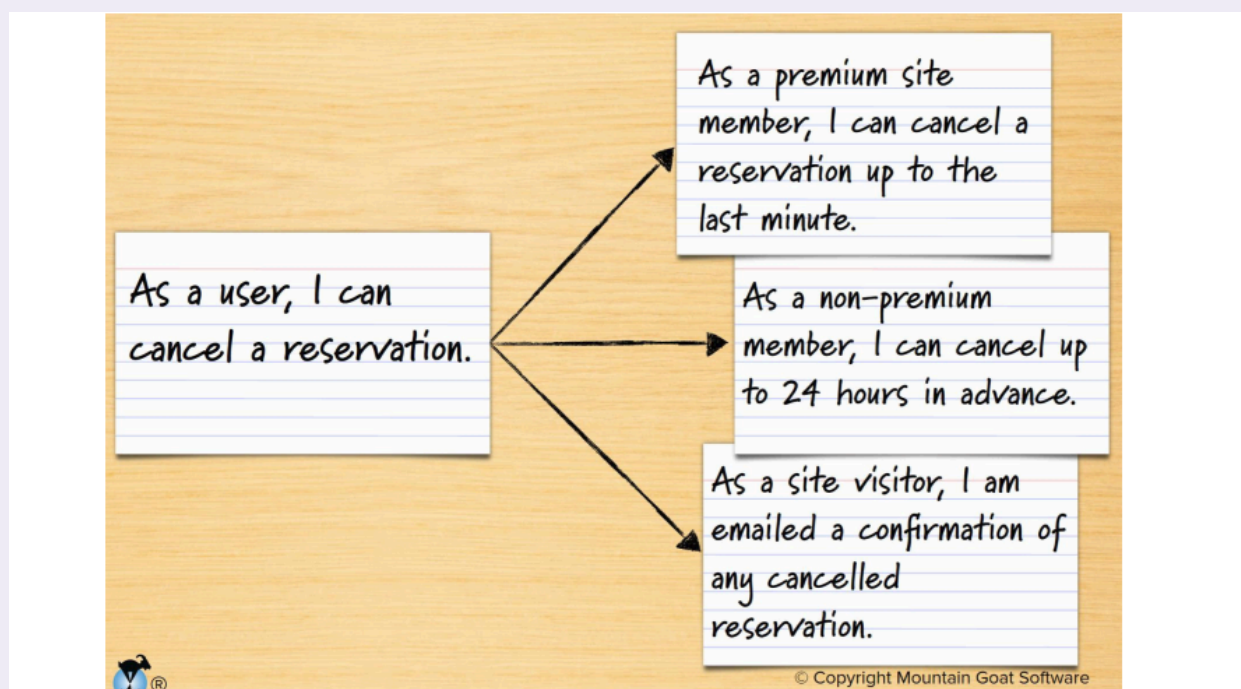
☰ Esempio di testi di accettazione e specifiche



Decomposizione di user story per iterazioni distinte

Nelle fasi di pianificazione, si manifesta frequentemente la necessità di decomporre una user story troppo generica al fine di poterne distribuire la realizzazione in iterazioni di sviluppo distinte

≡ Esempio di decomposizione di user story



Epic

Quando una user story si rivela essere di dimensioni troppo ampie per poter essere sviluppata e conclusa all'interno di una sola iterazione produttiva, essa assume formalmente la definizione di "Epic".

Un'Epic funge da contenitore tematico e deve inevitabilmente essere frammentata in entità minori.

☰ Esempio di epic

